ASSIGNMENT2 CO2

ASSIGNMENT 2 CO2

TASK 1 PRECEDENCE ASSOCIATIVITY AND DISCOUNT BUG

```
#include <stdio.h>

int main()
{
   int i;
   double price[3] = {100, 250, 80};
   int quantity[3] = {2, 1, 5};
   int senior[3] = {1, 0, 1};

   double total, discount, gst, net;
   double discountRate = 10.0;
   double gstRate = 5.0;

   int count = 0;

   printf("JAN AUSHADHI PHARMACY BILLING\n");
   printf("----------------------------------------\n");

   for(i = 0; i < 3; i++)
   {
      total = price[i] * quantity[i];

      if(senior[i] == 1)
         discount = total * discountRate / 100.0;
      else
         discount = 0.0;

      gst = (total - discount) * gstRate / 100.0;
```

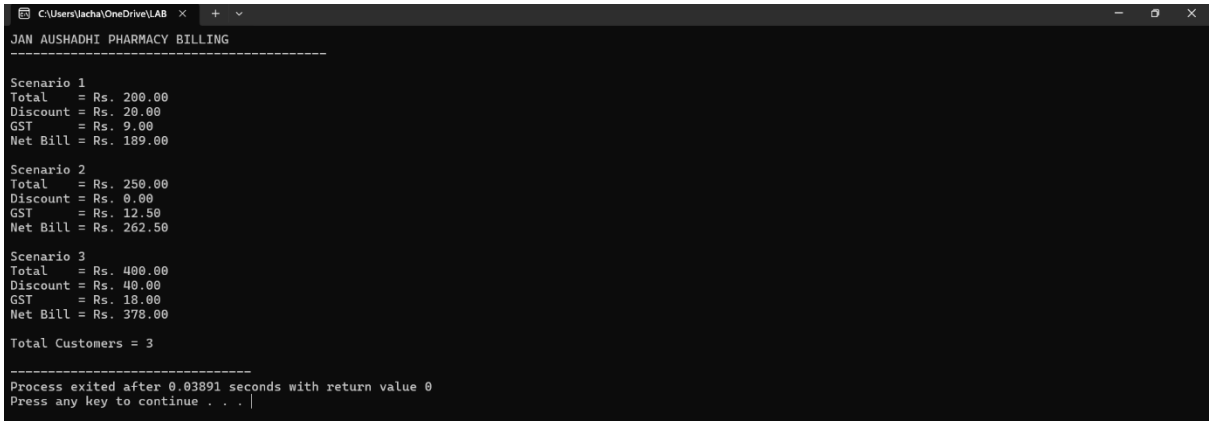
```
        net = (total - discount) + gst;

        count++;

        printf("\nScenario %d\n", i + 1);
        printf("Total    = Rs. %.2f\n", total);
        printf("Discount = Rs. %.2f\n", discount);
        printf("GST      = Rs. %.2f\n", gst);
        printf("Net Bill = Rs. %.2f\n", net);
    }

    printf("\nTotal Customers = %d\n", count);

    return 0;
}
```



TASK 2 WHEN DOES SHORT CIRCUIT EVALUATION MATTER

```
#include <stdio.h>

int calls = 0;

int verify()
{
    calls++;
```

```
    return 1;
}

int main()
{
    int age = 0, category = 1;

    if(age && verify())
        printf("Eligible\n");
    else
        printf("Not Eligible\n");

    printf("AND Calls = %d\n", calls);

    if(category || verify())
        printf("OR Short Circuit\n");

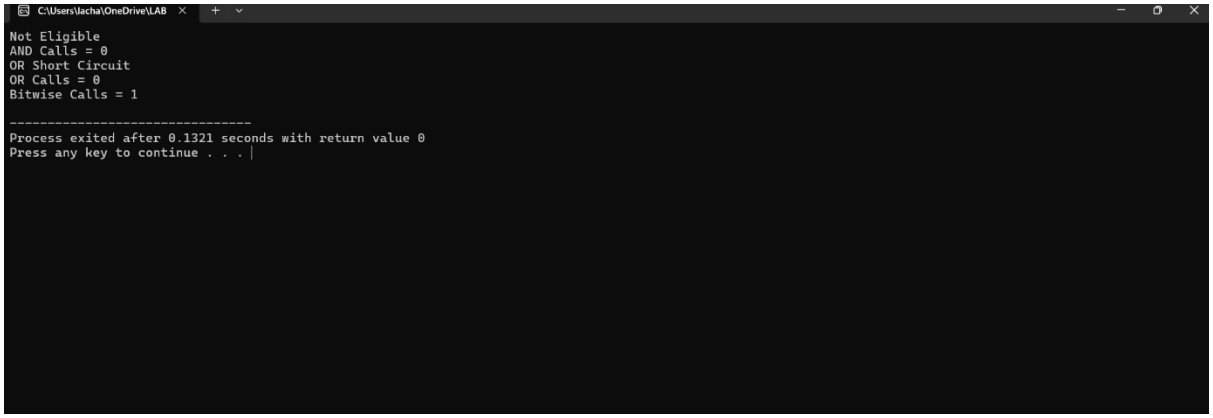
    printf("OR Calls = %d\n", calls);

    if(0 & verify())
        printf("True\n");

    printf("Bitwise Calls = %d\n", calls);

    return 0;
}
```

ASSIGNMENT2 C02



TASK 3 ENGINEER A DEVICE STATUS REGISTER WITH BITWISE OPERATORS

```
#include <stdio.h>

enum { LIGHT = 0, FAN = 1, AC = 2, HEATER = 3, PUMP = 4 };

unsigned char switch_on(unsigned char reg, int dev)
{
   return reg | (1 << dev);
}

unsigned char switch_off(unsigned char reg, int dev)
{
   return reg & ~(1 << dev);
}

unsigned char toggle(unsigned char reg, int dev)
{
   return reg ^ (1 << dev);
}

int is_on(unsigned char reg, int dev)
{
   return (reg & (1 << dev)) != 0;
```

```
}

void show(unsigned char reg)
{
    for(int bit = 7; bit >= 0; bit--)
        printf("%d", (reg >> bit) & 1);
    printf("\n");
}

int main()
{
    unsigned char reg = 0;

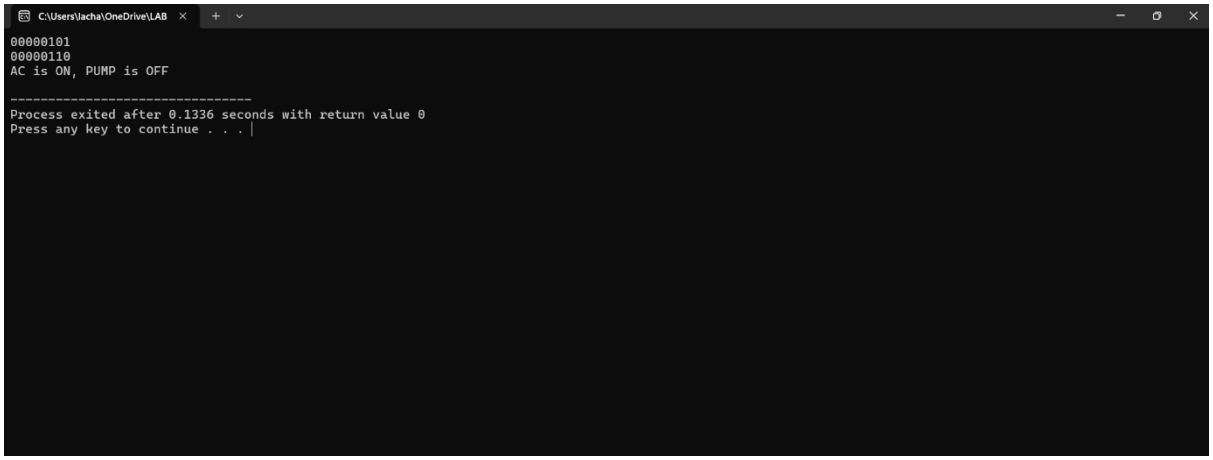
    reg = switch_on(reg, LIGHT);
    reg = switch_on(reg, AC);
    show(reg);

    reg = toggle(reg, FAN);
    reg = switch_off(reg, LIGHT);
    show(reg);

    printf("AC is %s, PUMP is %s\n",
        is_on(reg, AC) ? "ON" : "OFF",
        is_on(reg, PUMP) ? "ON" : "OFF");

    return 0;
}
```

ASSIGNMENT2 CO2



TASK 4 SWITCH

```
#include <stdio.h>

char grade(int marks)
{
   if(marks < 0 || marks > 100)
      return '?';

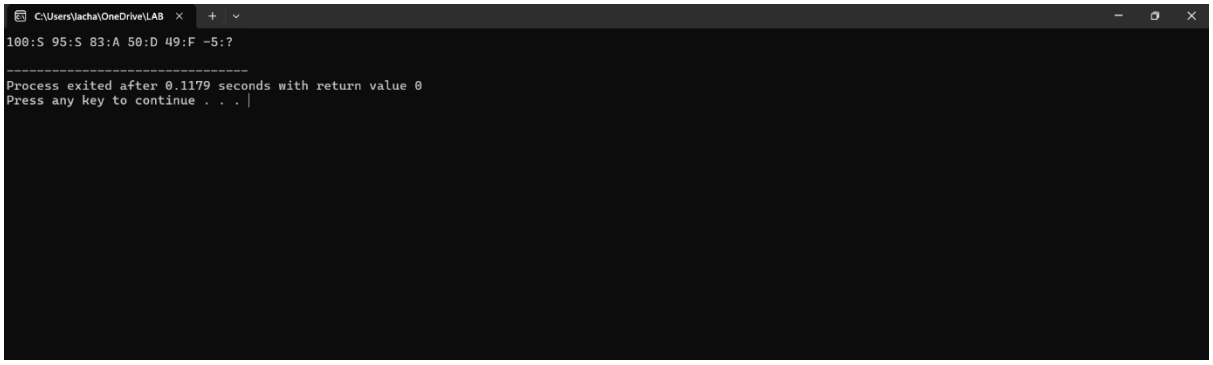
   switch(marks / 10)
   {
      case 10:
      case 9: return 'S';
      case 8: return 'A';
      case 7: return 'B';
      case 6: return 'C';
      case 5: return 'D';
      default: return 'F';
   }
}

int main(void)
{
```

```
    int marks[] = {100, 95, 83, 50, 49, -5};

    for(int i = 0; i < 6; i++)
        printf("%d:%c ", marks[i], grade(marks[i]));

    printf("\n");
    return 0;
}
```



TASK 5 COUNT THE WORK DONE BY A LOOP

```
#include <stdio.h>

unsigned long checks = 0;

int is_prime(unsigned long n)
{
    checks = 0;

    if(n < 2)
        return 0;

    for(unsigned long i = 2; i <= n / i; i++)
    {
        checks++;
```

```
        if(n % i == 0)
            return 0;
    }

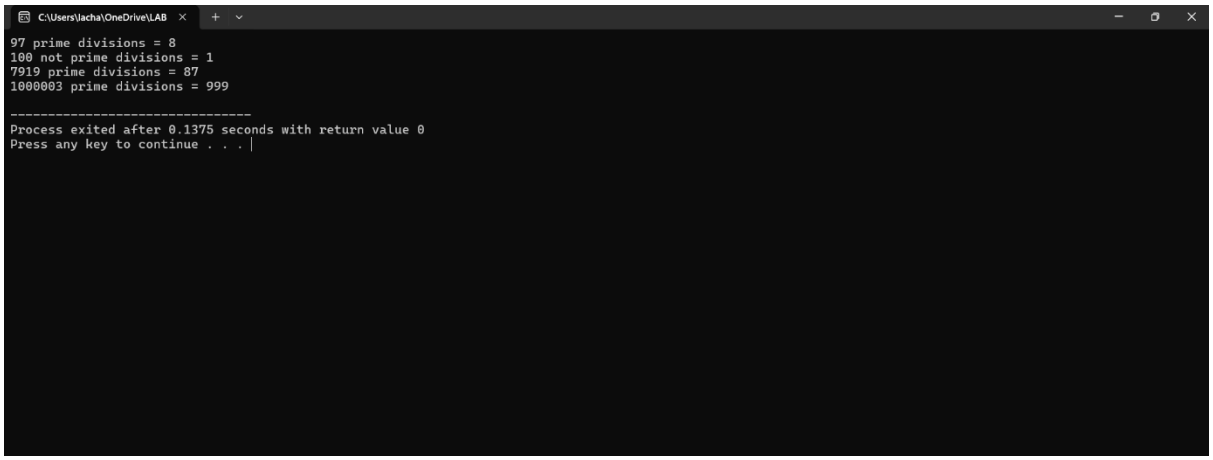
    return 1;
}

int main(void)
{
    unsigned long tickets[] = {97, 100, 7919, 1000003};

    for(int k = 0; k < 4; k++)
    {
        int p = is_prime(tickets[k]);

        printf("%lu %s divisions = %lu\n",
            tickets[k],
            p ? "prime" : "not prime",
            checks);
    }

    return 0;
}
```



ASSIGNMENT2 C02

TASK 6 LOOPS THAT MUST TERMINATE : LOAN REPAYMENT SCHEDULE

```
#include <stdio.h>

int main()
{
   double balance = 10000, rate = 0.02, payment = 2000;
   double interest, totalInterest = 0, paid;
   int month = 0;

   while (balance > 0 && month < 360)
   {
      month++;
      interest = balance * rate;
      totalInterest += interest;

      if (payment <= interest)
      {
         printf("Loan cannot be repaid\n");
         return 0;
      }

      paid = (balance + interest < payment) ?
            balance + interest : payment;

      balance = balance + interest - paid;

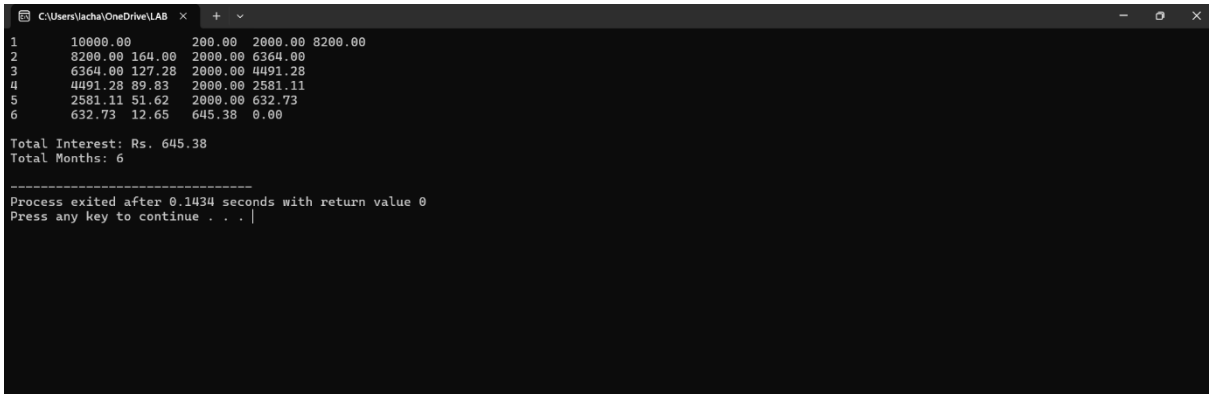
      if (balance < 0.005)
         balance = 0;

      printf("%d\t%.2f\t%.2f\t%.2f\t%.2f\n",
            month, balance + paid - interest,
```

```
            interest, paid, balance);
    }

    printf("\nTotal Interest: Rs. %.2f", totalInterest);
    printf("\nTotal Months: %d\n", month);

    return 0;
}
```



TASK 7 DIAGNOSE , REDESIGN AND VALIDATE

```
#include <stdio.h>

unsigned long original_count = 0;
unsigned long improved_count = 0;

int original_prime(unsigned long n)
{
    original_count = 0;

    if (n < 2)
        return 0;

    for (unsigned long i = 2; i < n; i++)
    {
        original_count++;
```

```
        if (n % i == 0)
            return 0;
    }

    return 1;
}

int improved_prime(unsigned long n)
{
    improved_count = 0;

    if (n < 2)
        return 0;

    if (n == 2)
        return 1;

    if (n % 2 == 0)
    {
        improved_count++;
        return 0;
    }

    improved_count++;

    for (unsigned long i = 3; i <= n / i; i += 2)
    {
        improved_count++;

        if (n % i == 0)
```

```
            return 0;
        }

        return 1;
    }

    int main()
    {
        unsigned long numbers[] = {
            97, 100, 7919, 1000003
        };

        printf("Number\tOriginal\tImproved\n");

        for (int i = 0; i < 4; i++)
        {
            unsigned long n = numbers[i];

            int a = original_prime(n);
            int b = improved_prime(n);

            printf("%lu\t%s (%lu)\t%s (%lu)\n",
                n,
                a ? "Prime" : "Not Prime",
                original_count,
                b ? "Prime" : "Not Prime",
                improved_count);
        }

        return 0;
    }
```

ASSIGNMENT2 CO2

